

Lab 6: Build a Real CI Pipeline Using Jenkins Scripted Pipeline

Project Scenario

ABC Software Pvt. Ltd. develops a simple Calculator application.

Whenever a developer pushes code to GitHub, Jenkins should automatically

- Download the latest code
- Install dependencies
- Run the application
- Execute unit tests
- Check coding standards
- Generate coverage report
- Store reports
- Notify whether build succeeded or failed

Today you are the DevOps Engineer responsible for building this CI pipeline.

Project Structure

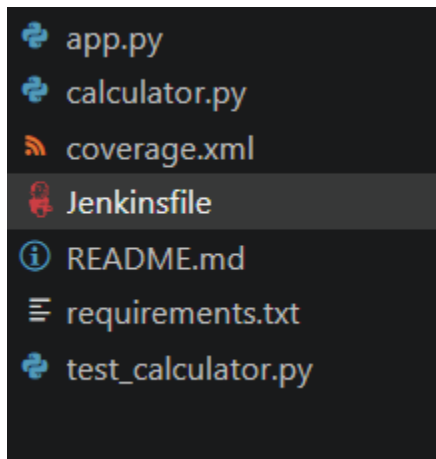
Create a folder

StudentManagementSystem

Inside it create

StudentManagementSystem

- |
- |— app.py
- |— calculator.py
- |— requirements.txt
- |— test_calculator.py
- |— Jenkinsfile
- |— README.md
- |— .gitignore
- |— reports



Step 1 Create calculator.py

```
calculator.py > add
1  def add(a, b):
2      return a + b
3
4
5  def subtract(a, b):
6      return a - b
7
8
9  def multiply(a, b):
10     return a * b
11
12
13 def divide(a, b):
14     return a / b
15
```

Step 2 Create app.py

```
app.py
1  from calculator import add, subtract, multiply, divide
2
3  print("Calculator Application")
4  print("Addition =", add(15, 10))
5  print("Subtraction =", subtract(15, 10))
6  print("Multiplication =", multiply(15, 10))
7  print("Division =", divide(15, 10))
```

```
D:\git and jira\Calculator_application>py app.py
Calculator Application
Addition = 25
Subtraction = 5
Multiplication = 150
Division = 1.5
```

Step 3 Create requirements.txt

pytest
coverage
flake8

```
requirements.txt
1  pytest
2  coverage
3  flake8
```

Step 4 Create Unit Test

```
test_calculator.py > ...
1  from calculator import add, subtract, multiply, divide
2
3
4  def test_add():
5      assert add(10, 5) == 15
6
7
8  def test_subtract():
9      assert subtract(10, 5) == 5
10
11
12 def test_multiply():
13     assert multiply(10, 5) == 50
14
15
16 def test_divide():
17     assert divide(10, 5) == 2
```

Step 5 Test Locally

Install packages: pip install -r requirements.txt

Run:- pytest

```
D:\git and jira\Calculator_application>pytest
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\git and jira\Calculator_application
collected 4 items

test_calculator.py .... [100%]

===== 4 passed in 0.05s =====
```

Step 6 Check Code Quality

Run: flake8 .

```
D:\git and jira\Calculator_application>flake8 .
D:\git and jira\Calculator_application>
```

Step 7 Generate Coverage

Run: coverage run -m pytest

Now generate report

coverage xml

A file

coverage.xml

```
coverage.xml
1 <?xml version="1.0" ?>
2 <coverage version="7.15.3" timestamp="178591931650" lines-valid="17" lines-covered="17" line-rate="1" branches-covered="0" branches-valid="0" branch-rate="0" complexity="0"
3 <!-- Generated by coverage.py: https://coverage.readthedocs.io/en/7.15.3 -->
4 <!-- Based on https://raw.githubusercontent.com/coverage-core/coverage/main/coverage-04.dtd -->
5 <sources>
6 <source source="D:\git and jira\Calculator_application\source">
7 </source>
8 </sources>
9 <packages>
10 <package name="." line-rate="1" branch-rate="0" complexity="0">
11 <classes>
12 <class name="calculator.py" filename="calculator.py" complexity="0" line-rate="1" branch-rate="0">
13 <methods>
14 <lines>
15 <line number="1" hits="1"/>
16 <line number="2" hits="1"/>
17 <line number="3" hits="1"/>
18 <line number="4" hits="1"/>
19 <line number="5" hits="1"/>
20 <line number="6" hits="1"/>
21 <line number="7" hits="1"/>
22 </lines>
23 </class>
24 <class name="test_calculator.py" filename="test_calculator.py" complexity="0" line-rate="1" branch-rate="0">
25 <methods>
26 <lines>
27 <line number="1" hits="1"/>
28 <line number="2" hits="1"/>
29 <line number="3" hits="1"/>
30 <line number="4" hits="1"/>
31 <line number="5" hits="1"/>
32 <line number="6" hits="1"/>
33 <line number="7" hits="1"/>
34 <line number="8" hits="1"/>
35 </lines>
36 </class>
37 </classes>
38 </package>
39 </packages>
40 </coverage>
```

Step 8 Initialize Git

git init

git add .

git commit -m "Completed Calculator Project"

Repository Name: student-management-system

git remote add origin https://github.com/ShwetaUbale/Calculator_application.git

git branch -M main

git push -u origin main

Step 9 Create Jenkinsfile

Create

Jenkinsfile

```

1 pipeline {
2   agent any
3
4   stages {
5     stage('Install dependencies') {
6       steps {
7         script {
8           if (isUnix()) {
9             sh '''
10             set -e
11             printf '%s\n' "${command -v python3 || command -v python || true}"
12
13             if [ -z "$PYTHON" ]; then
14               echo "No Python executable found"
15               exit 1
16             fi
17
18             # Create virtual environment
19             "python3" -m venv .venv
20
21             # Upgrade pip and install required packages inside .venv
22             .venv/bin/python -m pip install --upgrade pip
23             .venv/bin/python -m pip install -r requirements.txt
24             ...
25           } else {
26             bat 'python -m venv .venv'
27             bat ".venv\Scripts\python.exe -m pip install --upgrade pip"
28             bat ".venv\Scripts\python.exe -m pip install -r requirements.txt"
29           }
30         }
31       }
32     }
33
34     stage('Run application') {
35       steps {
36         script {
37           if (isUnix()) {
38             sh '.venv/bin/python app.py'
39           } else {
40             bat ".venv\Scripts\python.exe app.py"
41           }
42         }
43       }
44     }
45
46     stage('Run tests') {
47       steps {
48         script {
49           if (isUnix()) {
50             sh '.venv/bin/python -m coverage run -m pytest --junitxml=pytest-results.xml'
51           } else {
52             bat ".venv\Scripts\python.exe -m coverage run -m pytest --junitxml=pytest-results.xml"
53           }
54         }
55       }
56     }
57
58     stage('Generate coverage report') {
59       steps {
60         script {
61           if (isUnix()) {
62             sh '.venv/bin/python -m coverage xml'
63           } else {
64             bat ".venv\Scripts\python.exe -m coverage xml"
65           }
66         }
67       }
68     }
69
70     stage('Run lint') {
71       steps {
72         script {
73           if (isUnix()) {
74             sh '.venv/bin/python -m flake8 --exclude=.venv'
75           } else {
76             bat ".venv\Scripts\python.exe -m flake8 --exclude=.venv"
77           }
78         }
79       }
80     }
81
82   }
83
84   post {
85     always {
86       archiveArtifacts artifacts: 'coverage.xml, pytest-results.xml', allowEmptyArchive: true
87     }
88     success {
89       echo "Build, tests, linting, and coverage report completed successfully!"
90     }
91     failure {
92       echo "Pipeline build failed. Check stage logs above for details."
93     }
94   }
95 }

```

Step 10 Install Jenkins Plugins



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Step 11 Create Jenkins Pipeline

The screenshot shows the Jenkins web interface for a pipeline named 'Calculator_App'. At the top, a table lists pipeline runs with columns for status (S), name (W), last success, last failure, and last duration. Below this, the left sidebar contains navigation links: Status, Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main content area shows the 'Calculator_App' pipeline status as 'Success' with a green checkmark. It displays 'Last Successful Artifacts' as 'coverage.xml' (1.48 KiB) and 'pytest-results.xml' (537 B). A 'Permalinks' section lists various build links. At the bottom left, a 'Builds' dropdown menu is open, showing a list of builds with a filter input and a 'Today' section containing build #2 (14:31) and build #1 (14:25).

Step 12 Observe Console Output

```
+ .venv/bin/python app.py
Calculator Application
Addition = 25
Subtraction = 5
Multiplication = 150
Division = 1.5
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run tests)
[Pipeline] script
[Pipeline] {
[Pipeline] isUnix
[Pipeline] sh
+ .venv/bin/python -m coverage run -m pytest --junitxml=pytest-results.xml
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /var/jenkins_home/workspace/Calculator_App
collected 4 items

test_calculator.py ....
```

[100%]


```
- generated xml file: /var/jenkins_home/workspace/Calculator_App/pytest-
results.xml -
===== 4 passed in 0.04s =====
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Generate coverage report)
[Pipeline] script
[Pipeline] {
[Pipeline] isUnix
[Pipeline] sh
+ .venv/bin/python -m coverage xml
Wrote XML report to coverage.xml
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run lint)
[Pipeline] script
[Pipeline] {
[Pipeline] isUnix
[Pipeline] sh
+ .venv/bin/python -m flake8 . --exclude=.venv
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] echo
Build, tests, linting, and coverage report completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```

Finished: SUCCESS